Fatec
Faculdade de Tecnologia

Documentação de Software

Prof. Me. Deivison S. Takatu

deivison.takatu@fatec.sp.gov.br

Sumário

- Definição de Documentação de Software;
- Documentação e DevOps;
- Combinação de Áreas;
- Tipos de documentação;
- Github - README;
- Documentação do processo DevOps;
- Atividade.

Documentação de Projetos DevOps

- Se eu entrar hoje no GitHub do seu projeto, consigo entender e executar o projeto sem perguntar nada para vocês?
- A documentação deve reduzir a dependência de explicações informais e permitir que outra pessoa compreenda o projeto.
- **Exemplo**: github.com/deivisontakatu/aula-documentacao-software

O que acontece sem documentação?

- Dependência de desenvolvedores específicos;
- Dificuldade para novos integrantes entrarem no projeto;
- Configurações esquecidas ou executadas de forma diferente;
- Maior tempo para manutenção e resolução de problemas;
- Conhecimento perdido quando uma pessoa deixa a equipe;
- Perguntas repetitivas sobre instalação, execução e deploy.

O que é documentação de software

- Documentação de software é o conjunto de informações que ajuda pessoas a compreender, utilizar, desenvolver, testar e manter um sistema.
- Uma boa documentação responde: O que é o projeto? Para que ele serve? Como instalar e configurar? Como executar e testar? Como contribuir e realizar mudanças?

Por que documentar um projeto?

- A documentação apoia quatro necessidades principais:
  - Comunicação - compartilhar conhecimento entre pessoas
  - Manutenção - facilitar correções e evolução
  - Colaboração - permitir trabalho conjunto
  - Continuidade - reduzir a dependência de indivíduos
- Documentar não é apenas registrar, é tornar o conhecimento reutilizável.

Exemplos de Projetos Bem Documentados

- https://github.com/react/react
- https://github.com/nodejs/node
- https://github.com/docker/docs
- https://github.com/bitcoin/bitcoin
- https://github.com/github/brasil

Documentação e DevOps

- DevOps integra pessoas, processos e tecnologia para entregar software de forma contínua. A documentação contribui ao registrar:
  - como o código é desenvolvido;
  - como mudanças são validadas;
  - como a pipeline funciona;
  - como o sistema é implantado.

Quem utiliza a documentação?

- Dev - inicia o trabalho com menor dependência da equipe;
- Tester - identifica como executar e validar os testes;
- DevOps - configura pipeline, ambientes e deploy;
- Gestor - acompanha processos e decisões;
- Usuário - aprende a utilizar o produto.



Combinação de Áreas

- Engenharia de Software (ES) → requisitos, arquitetura, código e testes
- Gestão de Projetos (GP) → escopo, cronograma, riscos e entregas
- DevOps → CI/CD, infraestrutura, deploy e monitoramento
- Segurança da Informação → vulnerabilidades, políticas, acessos e incidentes
- Qualidade de Software (QA) → testes, critérios de aceitação, evidências e qualidade
- Produto / UX → requisitos do usuário, jornadas, regras de negócio e usabilidade

Tipos de documentação

- Documentação técnica - descreve arquitetura, APIs, banco, componentes e configurações.
- Documentação de projeto - registra objetivos, requisitos, decisões e organização.
- Documentação de processo - descreve branches, commits, Pull Requests, testes, pipeline e deploy.

README

- O README normalmente é o primeiro documento consultado em um repositório.
- Ele deve permitir que uma pessoa entenda rapidamente: propósito do projeto; principais tecnologias; pré-requisitos; instalação e execução; testes; contribuição; e links importantes.

README - Tecnologias e pré-requisitos

- Informe a stack necessária para executar o projeto.
- **Exemplos**: linguagem e versão; framework; banco de dados; gerenciador de pacotes; Docker, quando utilizado; Git e outras ferramentas.
- Indique versões mínimas ou recomendadas quando isso fizer diferença.

README - Instalação e execução

- A documentação deve transformar o processo em uma sequência reproduzível.
- Exemplo de fluxo: 1. Clonar o repositório; 2. Acessar a pasta; 3. Instalar dependências; 4. Configurar variáveis; 5. Executar o projeto; 6. Acessar a aplicação; 7. Executar os testes.

README - Configuração e variáveis

- Explique quais configurações são necessárias sem expor informações sensíveis. Exemplo: quais variáveis existem; para que cada variável serve; quais são obrigatórias; como criar o arquivo de configuração; quais valores podem ser utilizados como exemplo.

```
PORT = 3000
DATABASE_URL = sua_url_aqui
API_KEY = sua_chave_aqui
```

README - Execução e testes

- Uma documentação útil mostra como verificar se o projeto está funcionando.
- Inclua comandos para: iniciar a aplicação; executar testes unitários; executar testes de integração; executar testes de performance, quando houver; verificar resultados e falhas.

README - Estrutura do projeto

- Apresente a organização das principais pastas e arquivos.
- Exemplo: src/ para aplicação, tests/ para testes, docs/ para documentação e .github/workflows/ para pipelines.
- Explique a responsabilidade das partes relevantes.
- Evite documentar cada arquivo se isso não ajudar o leitor a compreender ou operar o sistema.

Documentação do processo DevOps

- Desenvolvimento: implementação de funcionalidades e correções no código.
- Git: controle de versão e histórico das alterações.
- Pull Request: solicitação de revisão e integração das mudanças.
- Build: compilação ou preparação do software para execução.

Documentação do processo DevOps

- Testes: validação do funcionamento e identificação de falhas.
- Pipeline CI/CD: automação das etapas de integração, validação e entrega.
- Deploy: publicação da aplicação em um ambiente de execução.

Tecnologias utilizadas

- Stack: conjunto de tecnologias utilizadas no desenvolvimento, como linguagens, frameworks, bancos de dados e ferramentas.
- Versões e ferramentas: registro das versões utilizadas para facilitar a compatibilidade e a reprodução do ambiente.
- Pré-requisitos: recursos necessários antes de executar o projeto, como runtimes, gerenciadores de pacotes, banco de dados, Git e Docker, quando aplicável.

Instalação e Execução

- Dependências e configuração inicial: instruções para instalar os pacotes necessários, configurar variáveis de ambiente e preparar serviços externos, como bancos de dados.
- Como iniciar a aplicação: sequência de comandos para executar o projeto e acessar a aplicação.
- Validação do ambiente: orientações para identificar erros comuns e confirmar que a aplicação está funcionando corretamente.

O que NÃO colocar na documentação

- Senhas e credenciais: nunca registrar senhas de usuários, administradores ou bancos de dados.
- Chaves de API e tokens: não expor credenciais que permitam acessar serviços, sistemas ou recursos privados.
- Dados sensíveis: evitar informações pessoais, dados confidenciais de clientes e informações internas restritas.

O que NÃO colocar na documentação

- Arquivos de configuração sigilosos: não publicar arquivos .env ou configurações que contenham segredos.
- Informações de infraestrutura restritas: não divulgar detalhes que facilitem acessos indevidos ou comprometam a segurança do sistema.

Atividade

Escolha um projeto de software e elabore sua documentação, apresentando o objetivo, o problema que pretende resolver, o escopo e os principais requisitos funcionais e não funcionais.

Descreva as tecnologias utilizadas, a arquitetura da aplicação e a organização dos principais componentes do sistema.

Atividade

Crie ou atualize o arquivo README.md do repositório no GitHub, incluindo os pré-requisitos, as instruções de instalação, configuração e execução do projeto.

Quando aplicável, documente também as APIs, o banco de dados, as variáveis de ambiente e os procedimentos de testes e deploy.

Referências

- HUMBLE J; PRIKLANDNICKI R. Entrega Contínua: Como Entregar Software de Forma Rápida e Confiável. São Paulo: Bookman, 2013
- MUNIZ, A.; et al. Jornada Devops: Unindo Cultura Ágil, Lean e Tecnologia Para Entrega De Software Com Qualidade. São Paulo: Brasport, 2019.
- SATO D. DevOps na prática: entrega de software confiável e automatizada. São Paulo: Casa do Código, 2014.
- SILVA, R. Entrega contínua em Android: Como automatizar a distribuição de apps. São Paulo: Casa do Código, 2016.
- ARUNDEL, J. DOMINGUS, J. DevOps nativo de nuvem com Kubernetes. São Paulo: Novatec, 2019.

Referências

- MORAES, G. Caixa de Ferramentas DevOps: Um guia para construção, administração e arquitetura de sistemas modernos. São Paulo: Casa do Código, 2015.
- PIRES, A.; MILITÃO, J. Integração Contínua com Jenkins. São Paulo: Casa do Código, 2019.
- VITALINO, J. F. N.; CASTRO, M. A. N. Descomplicando o Docker. 2 ed. São Paulo: Brasport, 2018. SILVERMAN, R. E. Git: guia prático. São Paulo: Novatec, 2019.
- KIM, G; HUMBLE, J; DEBOIS, P; WILLIS, J; Manual de DEVOPS: Como obter agilidade, confiabilidade e segurança em organizações tecnológicas. São Paulo: Starlin Alta Editora, 2018.

Fatec
Faculdade de Tecnologia

Documentação de Software

Prof. Me. Deivison S. Takatu

deivison.takatu@fatec.sp.gov.br